LAB 4 · LECTURE 4

Lecture 4 Lab: Benchmarking ANN Indexes with FAISS

Vector Databases Course | Estimated time: 60-75 minutes | Platform: Google Colab (CPU is enough)

Goal

You will build four FAISS indexes (Flat, HNSW, IVF-Flat, IVF-PQ) over the same 100,000 vectors and compare them on **recall@10**, **speed** and **size**. Flat search is exact, so it provides the ground truth for measuring recall.

- Measure how exact search cost grows with the dataset.
- Tune `efSearch` (HNSW) and `nprobe` (IVF) and observe the recall/speed trade-off.
- Estimate the memory savings of product quantization.
- Measure memory and tail latency (p95), and choose an index for a given workload.

Setup

Create a new Colab notebook and run each block below in its own cell, in order.

Cell 1: install and import

```
!pip install -q faiss-cpu numpy pandas matplotlib

import os, time
import numpy as np, pandas as pd
import faiss
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
```

Part 1: Data

We generate clustered synthetic vectors, which behave more like real embeddings than uniform random noise. (You can later replace them with sentence embeddings from Lecture 2.)

Cell 2: dataset

```
N, NQ, D, K = 100_000, 1_000, 128, 10

centers = rng.normal(size=(200, D)).astype("float32") * 3
def make(n):
    ids = rng.integers(0, len(centers), n)
    return (centers[ids] + rng.normal(size=(n, D))).astype("float32")

xb = make(N)    # database vectors
xq = make(NQ)   # query vectors
print(xb.shape, xq.shape)
```

Part 2: Ground truth and helpers

Cell 3: Flat index (exact) and benchmark helper

```
flat = faiss.IndexFlatL2(D)
flat.add(xb)
_, I_true = flat.search(xq, K)            # exact top-10 for every query

def recall_at_k(I):
    return float(np.mean([len(set(a) & set(b)) / K for a, b in zip(I, I_true)]))

def bench(index):
    t = time.perf_counter()
    _, I = index.search(xq, K)
    dt = time.perf_counter() - t
    return recall_at_k(I), NQ / dt, dt / NQ * 1000   # recall, QPS, ms/query

def size_mb(index):
    faiss.write_index(index, "tmp.index")
    return os.path.getsize("tmp.index") / 1e6

r, qps, ms = bench(flat)
print(f"Flat recall={r:.3f} QPS={qps:,.0f} size={size_mb(flat):.1f} MB")
```

Expected: recall is exactly 1.000. Note the QPS and size; they are your baseline.

YOUR TURN Run Cell 3 twice. Why do QPS values differ slightly between runs? Write one sentence.

Part 3: HNSW

Cell 4: build HNSW and sweep efSearch

```
t = time.perf_counter()
hnsw = faiss.IndexHNSWFlat(D, 32)          # M = 32 links per node
hnsw.hnsw.efConstruction = 200
hnsw.add(xb)
print(f"HNSW build time: {time.perf_counter() - t:.1f} s, size={size_mb(hnsw):.1f} MB")

rows = []
for ef in [4, 8, 16, 32, 64, 128, 256]:
    hnsw.hnsw.efSearch = ef
    r, qps, ms = bench(hnsw)
    rows.append(dict(index="HNSW", param=f"efSearch={ef}", recall=r, qps=qps, ms=ms))
pd.DataFrame(rows)
```

Part 4: IVF-Flat

Cell 5: build IVF-Flat and sweep nprobe

```
nlist = 1024                                # about 3 x sqrt(N)
quantizer = faiss.IndexFlatL2(D)
ivf = faiss.IndexIVFFlat(quantizer, D, nlist)
t = time.perf_counter()
ivf.train(xb)                               # k-means on the data
ivf.add(xb)
print(f"IVF build time: {time.perf_counter() - t:.1f} s, size={size_mb(ivf):.1f} MB")

for npb in [1, 2, 4, 8, 16, 32, 64]:
    ivf.nprobe = npb
    r, qps, ms = bench(ivf)
    rows.append(dict(index="IVF-Flat", param=f"nprobe={npb}", recall=r, qps=qps, ms=ms))
pd.DataFrame(rows).tail(7)
```

Part 5: IVF-PQ

Product quantization stores each 128-d vector as `m = 16` bytes instead of 512 bytes.

Cell 6: build IVF-PQ and sweep nprobe

```
m = 16                                      # sub-vectors; D must be divisible by m
quantizer2 = faiss.IndexFlatL2(D)
ivfpq = faiss.IndexIVFPQ(quantizer2, D, nlist, m, 8)   # 8 bits = 256 centroids per part
ivfpq.train(xb)
ivfpq.add(xb)
print(f"IVF-PQ size={size_mb(ivfpq):.1f} MB")

for npb in [1, 2, 4, 8, 16, 32, 64]:
    ivfpq.nprobe = npb
    r, qps, ms = bench(ivfpq)
    rows.append(dict(index="IVF-PQ", param=f"nprobe={npb}", recall=r, qps=qps, ms=ms))
```

Part 6: Memory arithmetic and tail latency

Cell 7: bytes per vector and p95 latency

```
bytes_flat = D * 4                                # float32
print(f"Flat   : {bytes_flat} B/vector")
print(f"IVF-PQ : {m} B/vector codes, {bytes_flat / m:.0f}x smaller")
print(f"HNSW   : +{2 * 32 * 4} B/vector of links at M=32")

hnsw.hnsw.efSearch = 64
lat = []
for q in xq[:300]:
    t0 = time.perf_counter()
    hnsw.search(q[None, :], K)
    lat.append((time.perf_counter() - t0) * 1000)
p50, p95, p99 = np.percentile(lat, [50, 95, 99])
print(f"p50 {p50:.2f} ms | p95 {p95:.2f} ms | p99 {p99:.2f} ms")
```

YOUR TURN Why can p95 be much larger than the average, and why does it matter for a user-facing search box? Compare the Flat and HNSW bytes per vector with the numbers from the saved index files.

Part 7: Compare

Cell 8: recall vs speed curve

```
df = pd.DataFrame(rows)
for name, g in df.groupby("index"):
    plt.plot(g["recall"], g["qps"], marker="o", label=name)
plt.yscale("log"); plt.xlabel("recall@10"); plt.ylabel("queries per second (log)")
plt.legend(); plt.grid(alpha=0.3); plt.title("Recall vs speed"); plt.show()
df.round(3)
```

YOUR TURN Record your measurements in the table below (pick the setting that first reaches recall of at least 0.90; write 'none' if the index never does).

Index	Setting for recall >= 0.90	QPS	Size (MB)	Build time (s)
Flat				
HNSW				
IVF-Flat				
IVF-PQ				

Part 8: Explore

- Set `M` to 8, then 64 for HNSW. How do build time, size and recall change?
- Try `nlist = 256` and `nlist = 4096` for IVF. Which needs a larger nprobe for the same recall?
- Increase N to 500,000 and re-measure Flat. Does the cost grow linearly, as the lecture predicts?
- Use inner product (`IndexFlatIP`) on normalized sentence embeddings instead of synthetic data.

Reflection questions

- Which index gave the best recall at the lowest latency, and what did it cost in memory and build time?
- Why does IVF-PQ plateau at a recall below 1.0 even with a very large nprobe?
- You must serve 200 million vectors on a machine with 64 GB of RAM. Which index would you start with, and why?
- Why is it important to measure recall on your own data rather than trusting a published benchmark?

Deliverable

Submit the executed notebook, the recall-vs-speed plot, the completed table, and short answers to the reflection questions.

Troubleshooting

- **Kernel runs out of memory:** reduce N to 50,000.
- **Absolute QPS differs from classmates:** Colab hardware varies; compare indexes against each other, not against other machines.
- **AssertionError on IVFPQ:** D must be divisible by m.